ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Challenge Set: Type Parameterization and Bounded Polymorphism

1. Diagnosing variance and bounds

(a) Does it compile?

```
trait Container[+A] { def add(x: A): Container[A] }
```

No: the type parameter `A` is declared covariant (`+A`), but the parameter `x: A` is an input position, The standard fix is the lower-bound trick:

```
// This says: I will accept any `B` that is at least as general as `A`,
// and return a `Container[B]` (a more general container).
trait Container[+A]:
  def add[B >: A](x: B): Container[B]
```

(b) does it compile? Can we fix it keeping it covariant?x

```
class Cell[+A](var content: A):
```

No, it does not compile: The parameter `A` is declared covariant (`+A`), but `var content: A` exposes `A` in both output position and input position.

No, we annot fix it: HWe can try and make `content` private and only accessible via the `get` and `set` method. On `set` we try the standard fix with lower bounds. It does not work:

```
class Cell[A](private var content: A):
  def get : A = content
  def set[U>:A] (v:U) : Unit = content = v
-- Type Mismatch Error: ------------------------------------------------
|  def set[U>:A] (v:U) : Unit = content = v
|                                       ^
|                    Found:    (v : U)
|                    Required: A
|                    where:    A is a type in class Cell with bounds <: U
|                              U is a type in method set with bounds >: A
```

(c) does it compile?

```
def maxOf[A <: Comparable[A]](xs: List[A]): A
```

Yes. This is an ordinary upper bound — in fact, an F-bound: the bound `Comparable[A]` mentions `A` itself. The body calls `x.compare(y)`, which typechecks because the bound guarantees that `A` has a `compare` method accepting another `A`. The result type `A`, rather than `Comparable[A]`, is preserved precisely because the bound is recursive.

(d) does it compile?

```
def firstSeq[A](xs: Set[A]): Seq[A] = xs
```

Yes. `Set[A]` and `Seq[A]` are *siblings* under `Iterable[A]`, not in a subtype relation. The compiler will not accept a `Set[A]` where a `Seq[A]` is expected. The fix depends on intent: weaken the result type to `Iterable[A]` (which both share), or convert explicitly with `xs.toSeq` (which constructs a new sequence, not a subtype coercion).

This question lives on the constructor-to-constructor axis: nominally, `Set` is not declared as a subtrait of `Seq`, and semantically it shouldn't be (`Set` guarantees uniqueness, `Seq` does not; `Seq` exposes indexing and
ordering, `Set` does not).

(e) Does it compile

```
trait Stack[+A]:
  def push[B >: A](x: B): Stack[B]
  def peek: A
```

Yes. Both methods type-check. `peek: A` is fine — `A` appears in output position, consistent with covariance. `push[B >: A](x: B): Stack[B]` is the classical trick to admit a method that conceptually consumes an `A` while keeping the trait covariant: the *method's own* type parameter `B` is a supertype of `A`, so `B` may appear in input position without violating variance, and the returned stack widens to `Stack[B]`.

2. Designing with F-bounds

Design a trait `Mergeable[A]` such that any type `A` extending `Mergeable[A]` provides a method `merge(that: A): A` for combining two values of the same concrete type. Then:

1. Implement two case classes that extend `Mergeable`: a `Counter(value: Int)` that merges by summing, and a `Log(entries: List[String])` that merges by concatenating.
2. Write a polymorphic function `mergeAll[A <: Mergeable[A]](xs: List[A]): Option[A]` that reduces a list using `merge`, returning `None` for the empty list.
3. Show — by attempting it and observing what the compiler says — what goes wrong if you drop the recursive bound and write `trait Mergeable` with `def merge(that: Mergeable): Mergeable`. Identify *which* concrete operation is no longer expressible.

Solution

```
trait Mergeable[A]:
  def merge(that: A): A

case class Counter(value: Int) extends Mergeable[Counter]:
  def merge(that: Counter): Counter = Counter(value + that.value)

case class Log(entries: List[String]) extends Mergeable[Log]:
  def merge(that: Log): Log = Log(entries ++ that.entries)

def mergeAll[A <: Mergeable[A]](xs: List[A]): Option[A] = xs match
  case Nil          => None
  case head :: tail => Some(tail.foldLeft(head)(_.merge(_)))
```

A quick check that the types behave as expected:

```
val c = mergeAll(List(Counter(1), Counter(2), Counter(3)))  // Option[Counter]
val l = mergeAll(List(Log(List("a")), Log(List("b"))))      // Option[Log]
```

The result is `Option[Counter]` and `Option[Log]` respectively, with the concrete type preserved — exactly what the F-bound buys.

The bound `A <: Mergeable[A]` on `mergeAll` does two jobs simultaneously:

1. It restricts the caller to types that know how to merge with values of their own concrete type.
2. It makes `head.merge(_)` typecheck, since `head: A` and `A` provides `merge(that: A): A`.

Using F-bounds in the trait. The trait may also be defined with a bound to qualify the parameter `A`, as in `trait Mergeable[A <: Mergeable[A]]`.

There are two motivations, of unequal weight.

The first is *documentation*. The bound on the trait expresses the invariant that this trait is intended to be implemented with `A` equal to the concrete extending type — ruling out (or at least flagging) oddities like `class WeirdCounter extends Mergeable[Log]`, which is syntactically valid but semantically nonsense.

The second, and the only one that is technically forcing, is when the trait itself contains *concrete* (non-abstract) methods that need to call back into `A`'s operations. For example:

```
trait Mergeable[A <: Mergeable[A]]:
  def merge(that: A): A
  def mergeThree(b: A, c: A): A =
         this.merge(b).merge(c)
         // merge(b) returns A; calling .merge(c) on A requires the
         // bound to tell us that A itself has merge
```

Without `A <: Mergeable[A]`, the expression `this.merge(b).merge(c)` does not typecheck: `this.merge(b)` has type `A`, and on a fully abstract `A` there is no `merge` method to call. The recursive bound is what makes `A`'s own operations visible *inside the trait*.

The general lesson: An F-bound can live in two places: on the trait, on a function that uses the trait, or both. Where it is *needed* depends on who needs to call `A`'s operations:

- If only an *external* function (`mergeAll`) calls `merge`, the bound on the function suffices.
- If the trait *itself* defines concrete methods that chain calls into `A`, the bound must also be on the trait.

This distinction is worth keeping in mind: the recursive bound is not a piece of decoration to be sprinkled on every self-referential trait, but a precise instrument whose placement should follow the actual structure of the code.

Dropping parametricity. Finally, we discuss the solution with recursive `Mergeable` s.

```
trait Mergeable:
  def merge(that: Mergeable): Mergeable

case class Counter(value: Int) extends Mergeable:
  def merge(that: Mergeable): Mergeable = that match
    case Counter(v) => Counter(value + v)
    case _          => throw new IllegalArgumentException("incompatible")
```

Two things break, both worth pointing out:

- **In the trait,** `merge` accepts *any* `Mergeable` , not just another `Counter` . So the implementation must either pattern-match (as above) or cast — losing static guarantees and admitting nonsense like `Counter(1).merge(Log(List("x")))` , which only fails at runtime.
- **In** `mergeAll` **,** the type of `head.merge(_)` is `Mergeable` , not `A` . Even if every individual call returns a `Counter` , the fold accumulates as `Mergeable` , and `Some(...)` has type `Option[Mergeable]` . The caller has lost track of the concrete type.

The single operation that becomes inexpressible is precisely *"merge two values of the same concrete type and get a value of that same type back."*
This is what the recursive bound recovers — the bound is not decoration but the very mechanism that ties argument and return type to a common, concrete A`.

3. A sorted collection hierarchy

Design a small parameterized hierarchy of immutable ordered collections:

- A top-level trait `Ordered[+A]` exposing only `def head: A` and `def tail: Ordered[A]` , sufficient to read elements in order.
- A subtrait `Insertable[A]` extending `Ordered[A]` that adds `def insert(x: A): Insertable[A]` . Decide the variance carefully — and justify it.
- Two concrete implementations: `SortedList[A]` (linked-list backing) and `SortedSet[A]` (binary search tree backing), each requiring `A` to admit an ordering.

Then answer:

1. Where did you need an upper bound, and what operations did the bound enable?
2. Why could `Ordered` be made covariant but `Insertable` could not — or, if you found a way to keep `Insertable` covariant, what trick did you use?
   (Hint: lower bounds on method type parameters, like `Stack.push` in Challenge 1.)
3. Does `SortedList[A] <: SortedSet[A]` hold? Should it? Justify your answer in terms of the operations and their semantics, not just the declared hierarchy.

Solution

```
trait Comparable[A <: Comparable[A]]:
  def compare(that: A): Int

trait Ordered[+A]:
  def head: A
  def tail: Ordered[A]

trait Insertable[A <: Comparable[A]] extends Ordered[A]:
  def insert(x: A): Insertable[A]

class SortedList[A <: Comparable[A]](xs: List[A]) extends Insertable[A]:
  def head: A = xs.head
  def tail: Ordered[A] = SortedList(xs.tail)
  def insert(x: A): Insertable[A] =
    val (smaller, bigger) = xs.span(_.compare(x) <= 0)
    SortedList(smaller ++ (x :: bigger))

class SortedSet[A <: Comparable[A]](xs: List[A]) extends Insertable[A]:
  def head: A = xs.head
  def tail: Ordered[A] = SortedSet(xs.tail)
  def insert(x: A): Insertable[A] =
    if xs.exists(_.compare(x) == 0) then this
    else
      val (smaller, bigger) = xs.span(_.compare(x) < 0)
      SortedSet(smaller ++ (x :: bigger))
```

Q1: where the upper bound is needed

To insert into a sorted collection, the implementation must *compare* values of `A`. This requires `A` to provide a comparison operation — exactly the situation that motivated bounded polymorphism in the

first place. We introduce a small trait for comparable values and use it as an F-bound:

The bound `A <: Comparable[A]` appears in two places: once on `Insertable`, to declare that any insertable collection must work with comparable elements; and once on each concrete implementation, propagating the constraint. Inside the body of `insert`, the bound enables the call `x.compare(y)`, exactly as the bound on `withResource` enabled `resource.close()`. This is the same pattern as `Mergeable` from Challenge 2: an F-bound declaring that `A` knows how to operate on values of its own type.

Q2: variance of `Ordered` vs `Insertable`

`Ordered` exposes `A` only in output positions (`head: A`, `tail: Ordered[A]`), so it can safely be covariant. An immutable `Ordered[Cat]` is freely usable as an `Ordered[Animal]`.

`Insertable[A]` adds `insert(x: A): Insertable[A]`, which puts `A` in input position. Covariance is therefore *unsound* — the same reason `Container[+A] { def add(x: A): ... }` failed in Challenge 1. There are two ways out:

- **Make `Insertable` invariant** — the simple choice. We lose the ability to use an `Insertable[S]` as an `Insertable[T]` for `S<:T` but we keep the ability to insert.
- **Keep it covariant using a lower bound on the method:**

```
trait Insertable[+A <: Comparable[A]] extends Ordered[A]:
  def insert[B >: A <: Comparable[B]](x: B): Insertable[B]
```

  Same trick as `Queue.enqueue`. The double constraint `B >: A <: Comparable[B]` also shows how lower and upper bounds can be combined
  on the same method type parameter.

Either answer is defensible if justified.

Q3: is `SortedList[A] <: SortedSet[A]` ?

No, and no. The relation does not hold (it is not declared, and Scala's nominal subtyping will not infer it), and it should not hold. `SortedList` and `SortedSet` are *implementation strategies* for the same abstract
contract `Insertable[A]`, not refinements of one another.